행복한 수열의 개수

1 이상 100 이하의 정수로만 이루어져 있는 N × N 크기의 격자 정보가 주어집니다.

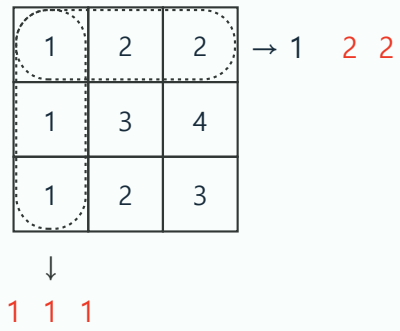
이때 행복한 수열은 다음과 같이 정의됩니다.

• 행복한 수열: 동일한 원소가 연속하여 M개 이상 나오는 구간이 존재하는 수열

각 행을 왼쪽에서 오른쪽으로 읽어 얻어지는 N개의 수열과 각 열을 위에서 아래로 읽어 얻어지는 N개의 수열, 총 2N개의 수열 중 행복한 수열의 개수를 세서 출력하는 프로그램을 작성해 주세요.

예를 들어, M = 2이고 격자가 다음과 같은 경우라면, 첫 번째 행을 골랐을 경우와 첫 번째 열을 골랐을 경우에만 행복한 수열이 되므로, 총 행복한 수열의 수는 2개가 됩니다.

codetree ▲



입력

첫 번째 줄에 격자의 크기 N과 연속해야 하는 원소의 수 M이 공백을 사이에 두고 주어집니다.

두 번째 줄부터 N개의 줄에 걸쳐 격자의 정보가 주어집니다. 이 중 i번째 줄에는 격자의 i번째 행에 위치한 N개의 정수 $A_{i,1}, A_{i,2}, \cdots, A_{i,N}$ 이 공백을 사이에 두고 주어집니다.

제약 조건

• $1 \le M \le N \le 100$
• $1 \le A_{i,j} \le 100$ $(1 \le i, j \le N)$

출력

2N개의 수열 중 행복한 수열의 개수를 출력합니다.

입출력 예시

예제 1

입력

```
3 2
1 2 2
1 3 4
1 2 3
```

출력

```
2
```

예제 2

입력

```
3 1
1 2 3
4 5 6
7 8 8
```

출력

```
6
```

제한

Python3 1000 ms / 128 MB

연속 구간 확인

접근 1 메인 - 정답

Intuition

가로 세로 2N개의 수열에 대해 행복한 수열인지 판단해서 결과를 반환합니다.

Algorithm

입력받은 격자의 행을 순회하며 각각의 행이 행복한 수열인지 판단하고 그 수를 셉니다. 그 후, 격자의 열을 순회하며 각각의 열이 행복한 수열인지 판단하고 그 수를 셉니다.

주어진 수열이 행복한 수열인지 판단하기 위해 함수 내에서 연속한 숫자의 수를 세는 변수를 사용합니다. 수열의 숫자들을 순회하며 현재 숫자와 이전 숫자가 같은 경우 연속한 숫자를 올려주고, 다른 경우 연속한 숫자의 수를 1로 초기화합니다. 이 숫자들 중 최댓값이 M 이상인 경우 참을 반환하고, 아닌 경우에는 거짓을 반환합니다.

Code

```python
# 변수 선언 및 입력:
n, m = tuple(map(int, input().split()))
grid = [
    list(map(int, input().split()))
    for _ in range(n)
]
seq = [0 for _ in range(n)]


def is_happy_sequence():
    # 주어진 seq가 행복한 수열인지 판단하는 함수입니다.
    consecutive_count, max_ccnt = 1, 1
    for i in range(1, n):
        if seq[i - 1] == seq[i]:
            consecutive_count += 1
        else:
            consecutive_count = 1

        max_ccnt = max(max_ccnt, consecutive_count)

    # 최대로 연속한 횟수가 m이상이면 true를 반환합니다.
    return max_ccnt >= m


num_happy = 0

# 먼저 가로로 행복한 수열의 수를 셉니다.
for i in range(n):
    seq = grid[i][:]

    if is_happy_sequence():
        num_happy += 1

# 세로로 행복한 수열의 수를 셉니다.
for j in range(n):
    # 세로로 숫자들을 모아 새로운 수열을 만듭니다.
    for i in range(n):
        seq[i] = grid[i][j]

    if is_happy_sequence():
        num_happy += 1

print(num_happy)
```

Complexity

- Time Complexity: $O(N^2)$
- Space Complexity: $O(N^2)$